Key Parameters' Posterior Sampling Time Analysis

Yifan CHENG y.cheng@u.nus.edu

Portions of Recorded Gibbs Sampler Time for 10 Key Parameters

We first display the first 50 kept post-burn-in MCMC iterations' posterior sampling time (in milliseconds) for 10 key Gibbs sampler steps corresponding to our 4 models, i.e., `fullGPfixedL`, `NNGPblockFixedL`, `NNGPsequenFixedL`, and `NNGPsequenVaryLj`.

```
wd <- paste(projDirec, "simu/mainScalabilityVerificationSimu/m3600T100K5", sep = "/")
setwd(wd)
load("GibbsStepTimeFixedLfullGP.RData")
load("GibbsStepTimeFixedLblock.RData")
load("GibbsStepTimeFixedLsequen.RData")
load("GibbsStepTimeVaryLjSequen.RData")
head(GibbsStepTimeFixedLfullGP, 50)
```

```
##          z  xi theta delta alpha kappa   rho eta upsilon psi
##  [1,] 3251 682   757    57 24175  2852 14345 800      55  57
##  [2,] 3221 675   762    58 24937  2863 17721 792      55  56
##  [3,] 3204 683   768    57 25752  2949 14249 847      55  57
##  [4,] 3196 683   771    57 24411  2917 17609 800      55  57
##  [5,] 3173 698   756    56 25320  2857 14176 815      55  57
##  [6,] 3183 691   754    57 24888  2898 17174 807      55  56
##  [7,] 3237 679   758    57 24866  2782 17641 823      54  57
##  [8,] 3241 688   772    57 25973  2947 17814 817      55  56
##  [9,] 3229 692   752    57 24819  2889 18083 801      54  56
## [10,] 3285 692   777    58 25417  2760 18350 817      54  56
## [11,] 3292 684   760    58 24744  2913 13878 827      55  57
## [12,] 3273 701   745    57 24530  2756 14348 816      55  56
## [13,] 3272 693   767    58 25036  2662 18338 809      55  57
## [14,] 3222 684   765    57 25488  2895 19373 838      55  57
## [15,] 3234 681   753    58 24469  2685 19014 818      55  57
## [16,] 3234 683   757    58 25075  2739 13981 823      55  56
## [17,] 3269 678   757    58 24779  2918 18505 814      54  56
## [18,] 3263 679   743    58 25907  2909 18105 820      55  57
## [19,] 3222 676   758    57 24662  2784 17868 806      55  57
## [20,] 3279 697   772    58 25303  2922 15330 818      54  56
## [21,] 3289 679   783    58 24682  2599 17275 830      55  56
## [22,] 3247 681   787    57 25779  2887 17167 855      56  56
## [23,] 3352 684   761    57 25049  2755 18769 828      55  57
## [24,] 3275 682   792    59 25056  2700 18002 873      56  57
## [25,] 3246 679   775    57 24419  2756 18896 840      56  57
## [26,] 3281 685   773    57 24591  2719 17284 822      54  57
## [27,] 3262 691   780    57 25298  2862 17160 828      55  57
## [28,] 3260 678   762    56 26654  2925 19818 845      57  59
## [29,] 3237 684   770    58 24548  2657 14235 827      57  58
```

```
## [30,] 3262 679   790    57 25017  2943 14715 817      55  56
## [31,] 3247 679   774    57 25466  2950 18271 842      56  57
## [32,] 3241 675   788    57 25974  2741 17772 830      56  58
## [33,] 3271 677   763    57 25674  2941 18537 812      55  57
## [34,] 3238 685   769    57 24335  2879 18136 832      56  59
## [35,] 3261 679   762    57 25346  2740 18213 826      56  57
## [36,] 3277 681   762    60 25651  2764 17736 817      55  59
## [37,] 3253 681   771    59 24829  2912 18499 822      56  58
## [38,] 3453 683   765    58 26493  2821 17982 830      55  57
## [39,] 3412 689   783    58 24283  2851 18255 827      55  58
## [40,] 3397 699   773    57 24676  2840 17509 819      57  58
## [41,] 3386 691   766    57 25973  2919 18823 829      55  57
## [42,] 3366 687   767    58 25848  2850 14591 822      55  57
## [43,] 3377 694   789    58 24913  2833 18838 840      57  57
## [44,] 3376 695   772    58 24803  2844 18160 811      54  56
## [45,] 3348 696   756    57 25216  2883 17961 810      54  57
## [46,] 3369 688   750    57 24927  2857 13949 832      57  59
## [47,] 3369 680   758    57 25894  2864 18320 817      53  55
## [48,] 3389 693   771    56 24489  2841 18274 825      54  56
## [49,] 3354 683   766    57 24891  2804 18052 812      53  56
## [50,] 3373 684   774    57 24667  2754 17790 819      55  56
```

```r
head(GibbsStepTimeFixedLblock, 50)
```

```
##          z  xi theta delta alpha kappa rho eta upsilon psi
##  [1,] 2695 706   780    60 22436   172 894 844      58  60
##  [2,] 2710 727   776    60 22328   175 906 846      59  60
##  [3,] 2697 718   780    62 22487   170 870 815      56  58
##  [4,] 2643 707   781    60 23223   166 875 839      57  59
##  [5,] 2589 682   785    57 21809   165 848 833      55  57
##  [6,] 2809 717   764    59 22087   169 870 799      56  58
##  [7,] 2710 710   783    60 22449   175 884 833      58  60
##  [8,] 2665 716   791    60 22498   176 897 845      58  60
##  [9,] 2569 690   780    60 21914   165 860 810      57  59
## [10,] 2665 687   756    59 21450   166 862 822      56  58
## [11,] 2768 729   789    60 23070   167 860 801      56  58
## [12,] 2792 743   793    58 23029   171 898 859      58  59
## [13,] 2627 713   770    59 22999   170 883 844      57  58
## [14,] 2752 724   798    59 21432   167 875 855      58  59
## [15,] 2658 714   768    58 22053   171 891 836      57  59
## [16,] 2686 733   796    59 22276   168 876 812      57  58
## [17,] 2719 747   800    62 23244   172 900 846      58  60
## [18,] 2678 708   768    61 21962   171 861 800      55  57
## [19,] 2614 681   763    58 21604   173 873 822      57  60
## [20,] 2820 716   785    59 22090   168 872 809      57  59
## [21,] 2772 744   786    60 22250   172 870 808      58  60
## [22,] 2726 727   793    62 23179   175 881 841      57  59
## [23,] 2726 716   776    58 22662   174 907 826      57  59
## [24,] 2720 726   770    59 22245   178 861 810      57  59
## [25,] 2713 720   772    60 22390   174 901 874      59  60
## [26,] 2703 744   797    60 22418   171 892 831      58  60
## [27,] 2744 697   788    60 22950   169 863 809      56  57
## [28,] 2685 705   817    59 21800   165 864 829      58  60
## [29,] 2813 696   774    60 22278   169 875 832      57  60
## [30,] 2677 690   781    59 22126   173 901 843      57  58
```

```
## [31,] 2666 730   793    60 22229   169 885 815      57  60
## [32,] 2547 693   770    58 21985   164 885 820      59  58
## [33,] 2666 698   780    58 21701   167 869 814      56  58
## [34,] 2744 732   798    60 23609   166 869 836      56  58
## [35,] 2698 726   778    59 23132   166 890 860      57  58
## [36,] 2677 735   780    59 22275   170 874 821      56  59
## [37,] 2615 705   762    57 21662   167 858 807      55  57
## [38,] 2691 703   770    58 21805   167 872 816      56  58
## [39,] 2713 704   756    59 22321   168 876 803      57  58
## [40,] 2727 726   777    60 22340   165 862 819      56  58
## [41,] 2617 716   767    59 22025   171 880 834      56  60
## [42,] 2754 703   792    60 22166   170 878 860      58  60
## [43,] 2700 711   788    60 22604   169 885 831      57  59
## [44,] 2675 740   788    59 22644   173 901 849      58  59
## [45,] 2732 745   810    60 22842   170 873 827      57  58
## [46,] 2735 693   777    58 22110   165 862 835      61  62
## [47,] 2664 702   764    58 21870   173 894 851      58  59
## [48,] 2616 713   782    59 22300   172 893 836      57  59
## [49,] 2709 726   786    60 22400   172 902 863      57  59
## [50,] 2680 710   817    60 21916   177 863 840      57  59
```

```r
head(GibbsStepTimeFixedLsequen, 50)
```

```
##          z  xi theta delta alpha kappa rho eta upsilon psi
##  [1,] 2752 676   797    58   933   160 801 834      54  56
##  [2,] 2679 678   797    57   951   164 809 849      55  57
##  [3,] 2673 678   794    58   940   161 808 835      55  56
##  [4,] 2763 692   794    58   938   164 804 846      55  57
##  [5,] 2696 685   795    59   938   161 791 833      55  57
##  [6,] 2671 687   801    59   943   165 814 852      55  57
##  [7,] 2685 692   805    58   945   162 811 860      55  57
##  [8,] 2709 688   796    58   941   163 817 840      55  56
##  [9,] 2787 686   799    59   947   163 815 863      56  57
## [10,] 2716 680   803    57   940   164 808 846      57  57
## [11,] 2704 680   792    57   942   161 809 869      55  57
## [12,] 2837 676   812    58   938   163 803 842      56  57
## [13,] 2678 675   800    58   947   161 807 859      55  57
## [14,] 2623 683   798    57   946   163 811 852      56  59
## [15,] 2714 675   810    58   951   168 813 849      55  57
## [16,] 2682 673   789    59   941   162 812 849      55  58
## [17,] 2744 676   790    57   937   164 816 844      56  59
## [18,] 2716 681   808    58   947   164 814 850      56  58
## [19,] 2733 683   811    63   949   166 807 848      56  58
## [20,] 2829 683   785    58   944   164 802 847      55  57
## [21,] 2715 683   796    59   950   164 811 847      56  57
## [22,] 2638 686   815    59   940   164 824 844      56  57
## [23,] 2686 689   793    58   942   164 806 841      56  56
## [24,] 2689 678   805    58   946   163 806 856      55  58
## [25,] 2752 677   801    58   942   161 809 844      56  58
## [26,] 2639 680   761    56   938   160 803 822      54  57
## [27,] 2704 683   787    57   939   167 825 855      56  58
## [28,] 2787 680   791    57   939   161 817 835      58  60
## [29,] 2683 693   815    59   948   161 799 828      56  57
## [30,] 2581 683   765    59   938   162 812 828      56  57
## [31,] 2661 687   800    58   937   161 811 843      57  60
```

```
## [32,] 2643 675   801    58   945   164 816 846      56  58
## [33,] 2770 680   814    59   933   158 800 818      53  56
## [34,] 2678 697   785    57   936   160 802 819      55  57
## [35,] 2663 680   807    57   937   163 805 835      53  55
## [36,] 2773 682   803    57   941   161 809 858      54  56
## [37,] 2655 685   814    57   946   160 805 854      55  57
## [38,] 2628 683   801    57   939   164 800 860      55  56
## [39,] 2688 677   767    58   939   161 805 829      54  57
## [40,] 2670 684   819    58   943   162 815 856      55  57
## [41,] 2783 682   786    63   949   162 802 817      56  55
## [42,] 2666 704   801    57   943   159 808 834      54  56
## [43,] 2655 675   782    56   935   158 789 830      53  55
## [44,] 2778 682   784    57   939   162 807 849      55  57
## [45,] 2670 681   819    58   937   165 811 834      55  57
## [46,] 2610 680   801    58   939   161 789 835      54  57
## [47,] 2691 681   782    57   943   160 814 838      54  56
## [48,] 2716 682   800    57   961   183 837 842      55  57
## [49,] 2744 680   800    58   938   162 805 824      54  56
## [50,] 2727 682   794    58   951   165 823 831      55  56
```

```r
head(GibbsStepTimeVaryLjSequen, 50)
```

```
##        u  xi theta delta alpha kappa rho eta upsilon psi
##  [1,] 57 100   777    55  2579   151 803 861      57  60
##  [2,] 58 101   804    57  2421   147 799 855      58  59
##  [3,] 58 103   814    57  2519   144 796 861      56  59
##  [4,] 57 102   801    58  2414   146 798 859      57  59
##  [5,] 56  97   786    56  2542   144 798 849      57  59
##  [6,] 56  99   781    56  2357   144 799 852      56  59
##  [7,] 58 100   800    59  2537   146 802 857      55  57
##  [8,] 58 100   787    58  2456   143 789 844      56  59
##  [9,] 55  97   802    56  2536   145 802 867      57  60
## [10,] 58 102   806    56  2381   143 779 814      54  56
## [11,] 57  96   769    57  2551   145 803 847      56  59
## [12,] 57  99   795    57  2390   141 792 839      56  59
## [13,] 57 100   788    56  2377   143 787 820      54  56
## [14,] 57  99   813    56  2438   145 806 858      57  59
## [15,] 57 100   801    56  2452   144 798 852      58  60
## [16,] 57  98   775    55  2385   144 801 862      56  58
## [17,] 57  99   807    57  2416   146 784 853      56  57
## [18,] 57  99   799    56  2417   143 802 863      57  59
## [19,] 56 100   791    56  2426   145 801 854      57  59
## [20,] 58 102   809    56  2397   143 793 843      56  57
## [21,] 58 104   786    57  2414   145 796 854      56  58
## [22,] 57 102   817    56  2448   149 806 878      57  59
## [23,] 57  98   783    56  2417   149 802 863      57  60
## [24,] 56  99   800    55  2412   146 791 856      56  58
## [25,] 56  98   798    56  2428   142 785 822      54  56
## [26,] 57 108   781    55  2392   139 791 846      55  57
## [27,] 56  99   789    56  2454   139 783 839      56  58
## [28,] 56  97   764    55  2425   141 786 836      56  57
## [29,] 55  95   774    54  2379   143 778 833      55  57
## [30,] 57  97   796    56  2399   142 767 833      54  56
## [31,] 56 100   788    56  2421   140 787 844      56  57
## [32,] 56 102   792    55  2447   142 788 820      56  57
```

```
## [33,] 57  95   787    55  2412   140 786 810      54  57
## [34,] 55  95   778    55  2400   141 780 817      57  57
## [35,] 56  97   796    56  2404   140 786 853      56  58
## [36,] 56  99   786    55  2405   143 787 834      55  56
## [37,] 56  97   801    56  2401   140 784 855      55  57
## [38,] 56  99   796    54  2402   141 790 839      55  57
## [39,] 55  93   780    55  2412   140 783 852      56  57
## [40,] 58  99   777    55  2392   141 795 848      55  57
## [41,] 56 105   800    57  2415   141 783 843      55  57
## [42,] 54  97   789    55  2403   144 793 832      55  57
## [43,] 55  96   792    55  2374   139 780 843      55  57
## [44,] 56 100   797    55  2383   143 784 820      54  55
## [45,] 57  99   791    56  2430   141 794 841      55  58
## [46,] 54  95   773    54  2378   143 785 831      55  57
## [47,] 56  97   786    55  2400   140 788 850      55  57
## [48,] 56  98   801    55  2433   141 785 825      54  55
## [49,] 55 100   789    55  2422   140 780 835      57  58
## [50,] 56  98   793    55  2412   140 792 849      56  57
```

As expected, there aren't any significant differences between our 4 models regarding posterior sampling time for the 3 temporal parameters ψ, Υ, and $\boldsymbol{\eta}_t$'s.

Posterior Sampling Time Summary Statistics

We then present vital posterior sampling time summary statistics for the 7 spatial-related parameters ($z_{jl}(\boldsymbol{s}_{i,o})$'s or $u_j(\boldsymbol{s}_{i,o})$'s, $\xi_j(\boldsymbol{s}_{i,o})$'s, θ_{jl_j}'s, $\delta_{1:k}$, ρ, κ, and $\alpha_{jl_j}(\boldsymbol{s}_{i,o})$'s) to showcase the manifest scalability improvements brought about by our 3 novelties, i.e., slice sampling, spatial NNGP, and sequential updates.

```r
apply(GibbsStepTimeFixedLfullGP[,1:7], 2, summary)
```

```
##                 z       xi    theta    delta    alpha    kappa      rho
## Min.     3004.000  661.000  735.000  55.0000 23299.00 2217.000 12856.00
## 1st Qu.  3314.000  688.000  800.000  58.0000 24641.25 2807.000 17131.50
## Median   4230.500  891.000 1007.000  70.0000 34192.50 3522.500 22529.50
## Mean     4460.977  909.047 1359.953  73.2364 35275.15 3607.779 24744.63
## 3rd Qu.  4943.000 1033.000 1842.000  82.0000 41689.25 4114.000 30622.25
## Max.    10498.000 1742.000 3607.000 185.0000 65573.00 6787.000 55795.00
```

```r
apply(GibbsStepTimeFixedLblock[,1:7], 2, summary)
```

```
##                z     xi    theta   delta    alpha    kappa      rho
## Min.    2442.000 670.00 723.0000 55.0000 21068.00 158.0000 827.0000
## 1st Qu. 2621.000 700.00 768.0000 58.0000 21927.00 168.0000 869.0000
## Median  2674.000 713.00 780.0000 59.0000 22268.50 170.0000 879.0000
## Mean    2677.307 713.57 779.6124 59.2832 22316.23 170.1582 879.0884
## 3rd Qu. 2732.000 726.00 791.0000 60.0000 22644.00 172.0000 889.0000
## Max.    2987.000 873.00 856.0000 66.0000 24386.00 204.0000 973.0000
```

```r
apply(GibbsStepTimeFixedLsequen[,1:7], 2, summary)
```

```
##                z       xi    theta   delta    alpha    kappa      rho
## Min.    2513.000 663.0000 737.0000 54.0000 927.0000 155.0000 771.0000
## 1st Qu. 2652.000 677.0000 778.0000 57.0000 938.0000 160.0000 798.0000
## Median  2681.000 681.0000 789.0000 57.0000 941.0000 161.0000 805.0000
## Mean    2687.017 681.1242 788.5526 57.5628 941.9378 162.2748 805.6914
## 3rd Qu. 2720.000 685.0000 799.0000 58.0000 945.0000 163.0000 812.0000
```

```
## Max.    2921.000 821.0000 875.0000 66.0000 973.0000 196.0000 877.0000
```

```r
apply(GibbsStepTimeVaryLjSequen[,1:7], 2, summary)
```

```
##               u       xi    theta   delta    alpha    kappa      rho
## Min.    52.0000  92.0000 742.0000 52.0000 2333.000 136.0000 754.0000
## 1st Qu. 55.0000  97.0000 777.0000 55.0000 2402.000 141.0000 782.0000
## Median  56.0000  99.0000 788.0000 55.0000 2418.000 142.0000 788.0000
## Mean    56.1756  99.2364 787.6832 55.5148 2442.089 142.3264 788.5346
## 3rd Qu. 57.0000 101.0000 797.0000 56.0000 2483.000 143.0000 794.0000
## Max.    67.0000 243.0000 854.0000 63.0000 2709.000 167.0000 852.0000
```

The results correspond well to what we have deduced in Appendix H of our manuscript.

- Compared to their `fullGPfixedL` counterparts, `NNGPblockFixedL`'s Gibbs sampler steps corresponding to ρ and κ are evidently accelerated by our **spatial NNGP prior**;
- The only Gibbs sampler step time that should clearly differ between `NNGPblockFixedL` and `NNGPsequenFixedL` is the step updating all $\alpha_{jl}(\boldsymbol{s}_{i,o})$'s, which result from whether we adopt our **sequential updating method** or not. Since $m = 3600$ here is big, `NNGPsequenFixedL` is more than 20 times faster than `NNGPblockFixedL` for the posterior sampling step corresponding to $\alpha_{jl}(\boldsymbol{s}_{i,o})$'s;
- Thanks to our **slice sampling approach**, `NNGPsequenVaryLj`'s Gibbs sampler steps for $u_j(\boldsymbol{s}_{i,o})$'s, $\xi_j(\boldsymbol{s}_{i,o})$'s, ρ, and κ are significantly faster than `NNGPsequenFixedL`'s Gibbs sampler steps for $z_{jl}(\boldsymbol{s}_{i,o})$'s, $\xi_j(\boldsymbol{s}_{i,o})$'s, ρ, and κ. It turns out that `NNGPsequenVaryLj`'s Gibbs sampler step for $\alpha_{jl_j}(\boldsymbol{s}_{i,o})$'s is slower than its `NNGPsequenFixedL` counterpart, indicating that inefficiencies caused by case discussion, calculating all required upper or lower bounds, and rejection sampling outweigh acceleration brought about by slice sampling's ensured non-increasing posterior samples for L_j's through the MCMC iterations.

We finally calculate standard deviations for the 7 spatial-related parameters' posterior sampling time across all kept post-burn-in MCMC iterations.

```r
round(apply(GibbsStepTimeFixedLfullGP[,1:7], 2, sd), 5)
```

```
##           z          xi       theta       delta       alpha       kappa
##  1385.28987   235.89833   684.80304    17.78304 11141.16179   947.57928
##         rho
##  9079.43828
```

```r
round(apply(GibbsStepTimeFixedLblock[,1:7], 2, sd), 5)
```

```
##         z        xi     theta     delta     alpha     kappa       rho
##  81.33616  18.24969  17.06282   1.27284 535.56258   3.70004  15.47451
```

```r
round(apply(GibbsStepTimeFixedLsequen[,1:7], 2, sd), 5)
```

```
##        z       xi    theta    delta    alpha    kappa      rho
## 54.95417  8.69054 16.23415  1.26838  5.92909  4.47505 12.09880
```

```r
round(apply(GibbsStepTimeVaryLjSequen[,1:7], 2, sd), 5)
```

```
##        u       xi    theta    delta    alpha    kappa      rho
##  1.20559  5.48869 15.03434  1.13773 60.05514  3.13040 10.14898
```

Data Generation and Model Fitting

```r
rm(list=ls())
library(mvtnorm)
library(fields)
```

```r
library(spatempBFA)
library(coda)
K <- 5
O <- 1
L <- 50
M <- 3600
LjVec <- rep(min(50, M), K)
sqrootM <- 60
Nu <- 100
Time <- 1:Nu
TimeDist <- as.matrix(dist(Time))
APsi = 0.1; BPsi = 4.5
set.seed(29)
### 1) actual sigma^2(i,o) (for i=1,2,...,M and o=1) values
sigma2 <- 0.01
### 2) actual psi value
psi <- 2.3
### 3) actual kappa value
kappa <- 0.7
### 4) actual Upsilon(K\times K)
tempMat <- matrix(runif(K*K,0,1),K,K)
Upsilon <- t(tempMat)%*%tempMat
### 5) actual rho value
rho <- 0.8
D <- rdist(expand.grid(1:sqrootM, 1:sqrootM))
Frho <- exp(-rho*D)
### 6) actual Eta (c(Eta_1,...,Eta_T)) (vec of length Nu*K)
Hpsi <- exp(-psi*TimeDist)
Eta <- rmvnorm(1, mean=rep(0, Nu*K), sigma=kronecker(Hpsi, Upsilon))
## Y~0 (P=0) so no need to sample Beta; all familyInd=0 (all normal) so no need to sample Y
maxL <- 10
LStarJ <- sample(maxL, size=K, replace=T)
### 7) actual alpha
Alpha <- list()
for(j in 1:K) {
  Alpha[[j]] <-  t(rmvnorm(LStarJ[j], mean=rep(0,M*O), sigma=kappa*Frho))
  #every list index an M by L_j matrix
}
w <- list()
for(j in 1:K){
  w[[j]] <- pnorm(Alpha[[j]])
  Lj <- LStarJ[j]
  w[[j]][,Lj] <- rep(1, M)
  temp <- rep(1, M)
  for(l in 1:Lj){
    w[[j]][,l] <- w[[j]][,l]*temp
    if(l<Lj) {temp <- temp * pnorm(Alpha[[j]][,l], lower.tail = FALSE)}
  }
}
### 8) actual Xi
Xi <- matrix(1, M, K)
for(j in 1:K){
  Lj <- LStarJ[j]
```

```r
  for(i in 1:M){
    Xi[i,j] <- sample(Lj, size=1, prob=w[[j]][i,])
  }
}
### 9) actual Delta
a1=1; a2=10
Delta <- sapply(c(a1,rep(a2,(K-1))), rgamma, n=1, rate=1)
#Tau <- cumprod(Delta)
### 10) actual Theta
Theta <- list()
for(j in 1:K){
  Theta[[j]] <- rnorm(LStarJ[j], 0, sd=sqrt(1/Delta[j])) #vector of length Lj
}
Lambda <- matrix(0, M*O, K)
for(j in 1:K){
  for(i in 1:M){
    Lambda[i,j] = Theta[[j]][Xi[i,j]]
  }
}
Sigma.NuMO <- rnorm(Nu * M * O, sd = sqrt(sigma2))
EtaMat <- matrix(Eta, K, Nu)
meanMat <- Lambda%*%EtaMat #M*O\times Nu
Yobs <- as.vector(meanMat) + Sigma.NuMO
dat <- data.frame(Y = Yobs)
Hypers <- list(Sigma2 = list(A = 1, B = 1), Rho = list(ARho=0.1, BRho=1),
               Kappa = list(SmallUpsilon = O + 1, BigTheta = diag(O)),
               Psi = list(APsi = APsi, BPsi = BPsi),
               Upsilon = list(Zeta = K + 1, Omega = diag(K)))
MCMC <- list(NBurn = 20000, NSims = 10000, NThin = 2, NPilot = 5)
regFixedL.simu <- bfaFixedL(Y ~ 0, data = dat, dist = D, time = Time,  K = K,
                            starting = NULL, hypers = Hypers, tuning = NULL, mcmc = MCMC,
                            L = L,
                            family = "normal",
                            temporal.structure = "exponential",
                            spatial.structure = "continuous",
                            seed = 29,
                            gamma.shrinkage = TRUE,
                            include.time = TRUE,
                            include.space = TRUE,
                            clustering = TRUE,
                            seasonPeriod = 1,
                            equalTimeDist = TRUE,
                            spatApprox = FALSE,
                            alphaMethod = "block",
                            h = 15,
                            storeSpatPredPara = FALSE,
                            storeWeights = FALSE,
                            alphasWeightsToFiles = FALSE)
save(regFixedL.simu, file="regFixedL50simuT100M3600Iter30000.RData")
Diags <- diagnostics(regFixedL.simu, diags = c("dic", "dinf", "meanIC", "waic"),
                     keepDeviance = TRUE)
save(Diags, file = "regFixedL50simuT100M3600Iter30000Diags.RData")
Deviance <- as.mcmc(Diags$deviance)
```

```r
save(Deviance, file = "regFixedL50simuT100M3600Iter30000Deviance.RData")
GibbsStepTimeFixedLfullGP <- regFixedL.simu$GibbsStepTime
save(GibbsStepTimeFixedLfullGP, file = "GibbsStepTimeFixedLfullGP.RData")
regFixedL.simu.block <- bfaFixedL(Y ~ 0, data = dat, dist = D, time = Time,  K = K,
                                  starting = NULL, hypers = Hypers, tuning = NULL,
                                  mcmc = MCMC,
                                  L = L,
                                  family = "normal",
                                  temporal.structure = "exponential",
                                  spatial.structure = "continuous",
                                  seed = 29,
                                  gamma.shrinkage = TRUE,
                                  include.time = TRUE,
                                  include.space = TRUE,
                                  clustering = TRUE,
                                  seasonPeriod = 1,
                                  equalTimeDist = TRUE,
                                  spatApprox = TRUE,
                                  alphaMethod = "block",
                                  h = 15,
                                  storeSpatPredPara = FALSE,
                                  storeWeights = FALSE,
                                  alphasWeightsToFiles = FALSE)
save(regFixedL.simu.block, file="regFixedL50simuBlockT100M3600Iter30000.RData")
Diags.block <- diagnostics(regFixedL.simu.block,
                           diags = c("dic", "dinf", "meanIC", "waic"),
                           keepDeviance = TRUE)
save(Diags.block, file = "regFixedL50simuBlockT100M3600Iter30000Diags.RData")
Deviance.block <- as.mcmc(Diags.block$deviance)
save(Deviance.block, file = "regFixedL50simuBlockT100M3600Iter30000Deviance.RData")
GibbsStepTimeFixedLblock <- regFixedL.simu.block$GibbsStepTime
save(GibbsStepTimeFixedLblock, file = "GibbsStepTimeFixedLblock.RData")
regFixedL.simu.sequen <- bfaFixedL(Y ~ 0, data = dat, dist = D, time = Time,  K = K,
                                   starting = NULL, hypers = Hypers, tuning = NULL,
                                   mcmc = MCMC,
                                   L = L,
                                   family = "normal",
                                   temporal.structure = "exponential",
                                   spatial.structure = "continuous",
                                   seed = 29,
                                   gamma.shrinkage = TRUE,
                                   include.time = TRUE,
                                   include.space = TRUE,
                                   clustering = TRUE,
                                   seasonPeriod = 1,
                                   equalTimeDist = TRUE,
                                   spatApprox = TRUE,
                                   alphaMethod = "sequential",
                                   h = 15,
                                   storeSpatPredPara = FALSE,
                                   storeWeights = FALSE,
                                   alphasWeightsToFiles = FALSE)
save(regFixedL.simu.sequen,
```

```r
     file="regFixedL50simuSequenT100M3600Iter30000nostorealphaweights.RData")
Diags.sequen <- diagnostics(regFixedL.simu.sequen,
                            diags = c("dic", "dinf", "meanIC", "waic"),
                            keepDeviance = TRUE)
save(Diags.sequen,
     file = "regFixedL50simuSequenT100M3600Iter30000DiagsNostorealphaweights.RData")
Deviance.sequen <- as.mcmc(Diags.sequen$deviance)
save(Deviance.sequen,
     file = "regFixedL50simuSequenT100M3600Iter30000DevianceNostorealphaweights.RData")
GibbsStepTimeFixedLsequen <- regFixedL.simu.sequen$GibbsStepTime
save(GibbsStepTimeFixedLsequen, file = "GibbsStepTimeFixedLsequen.RData")
regVaryLj.simu.sequen <- bfaVaryingLjs(Y ~ 0, data = dat, dist = D, time = Time,  K = K,
                                       LjVec = LjVec,
                                       starting = NULL, hypers = Hypers, tuning = NULL,
                                       mcmc = MCMC,
                                       family = "normal",
                                       temporal.structure = "exponential",
                                       spatial.structure = "continuous",
                                       seed = 29,
                                       gamma.shrinkage = TRUE,
                                       include.time = TRUE,
                                       include.space = TRUE,
                                       seasonPeriod = 1,
                                       equalTimeDist = TRUE,
                                       spatApprox = TRUE,
                                       alphaSequen = TRUE,
                                       h = 15,
                                       storeSpatPredPara = FALSE,
                                       storeWeights = FALSE)
save(regVaryLj.simu.sequen,
     file="regVaryLjsimuSequenT100M3600Iter30000nostorealphaweight.RData")
Diags.sequenVaryLj <- diagnostics(regVaryLj.simu.sequen,
                                  diags = c("dic", "dinf", "meanIC", "waic"),
                                  keepDeviance = TRUE)
save(Diags.sequenVaryLj,
     file = "regVaryLjsimuSequenT100M3600Iter30000DiagsNostorealphaweight.RData")
Deviance.sequenVaryLj <- as.mcmc(Diags.sequenVaryLj$deviance)
save(Deviance.sequenVaryLj,
     file = "regVaryLjsimuSequenT100M3600Iter30000DevianceNostorealphaweight.RData")
GibbsStepTimeVaryLjSequen <- regVaryLj.simu.sequen$GibbsStepTime
save(GibbsStepTimeVaryLjSequen,
     file = "GibbsStepTimeVaryLjSequen.RData")
```